# 🏥 Apex Horizon Hospital Management System - Complete Architectural & Operational Flowcharts

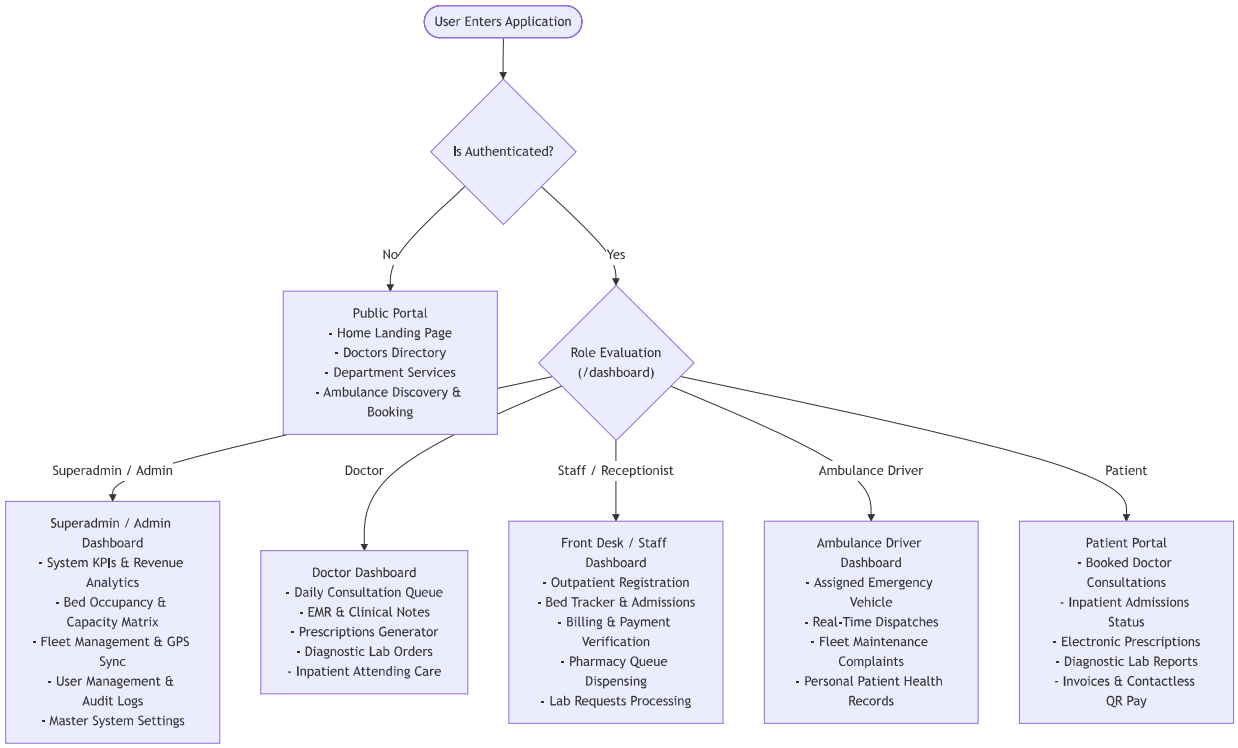
This document provides a comprehensive, fully functional flowchart and process-flow breakdown of the **Apex Horizon Medical Center** platform (built on Laravel 11 with multi-role RBAC, EMR, Inpatient ADT, Emergency Ambulance Fleet with GPS tracking, Pharmacy, Diagnostics, and Contactless QR Payments).

## 📑 Table of Contents


1. High-Level System Architecture & User Roles
2. Authentication, Authorization & Password Recovery (OTP) Flow
3. Outpatient Consultation & Patient EMR Lifecycle
4. Inpatient Bed Management & ADT Flow (Admission, Transfer & Discharge)
5. Emergency Ambulance Booking & Live GPS Tracking Flow
6. Pharmacy Inventory & Medication Dispensing Flow
7. Diagnostic Laboratory & Pathology Workflow
8. Billing, Contactless QR Payments & Cashier Verification Flow
9. Driver Duty & Ambulance Maintenance Complaint Flow
10. End-to-End Cross-Module Entity Relationship Flow

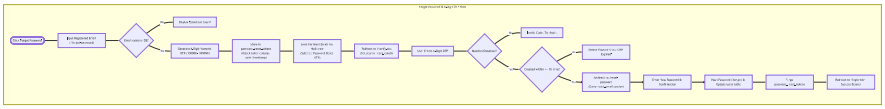

## 1. High-Level System Architecture & User Roles

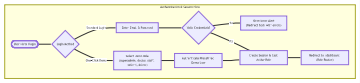
The system is organized around Role-Based Access Control (RBAC) where each user role has distinct permissions, workflows, and automated dashboard redirections.



## 2. Authentication, Authorization & Password Recovery (OTP) Flow

Includes standard credential login, role-specific demo logins, registration, and a secure 6-digit OTP verification password recovery mechanism.





## 3. Outpatient Consultation & Patient EMR Lifecycle

Outpatient care connects public requests, front-desk scheduling, doctor consultations, electronic medical records (EMR), prescription writing, and automatic billing invoice creation.

Mermaid render error: Failed to fetch dynamically imported module:
https://markdownlivepreview.com/assets/sequenceDiagram-SI44F4Z6-Cc9Aa9NS.js

## 4. Inpatient Bed Management & ADT Flow (Admission, Transfer & Discharge)

The Admission, Discharge, and Transfer (ADT) engine manages hospital rooms, beds, and real-time clinical state transitions with strict role authorizations.

```
                                flowchart TD
                        subgraph BedStates["Bed State Life-Cycle"]
                                    B_Avail[Available Bed]
                                     B_Occ[Occupied Bed]
                                B_Clean[Cleaning / Sanitization]
                                    B_Maint[Maintenance]

                                B_Avail -->|Patient Admitted| B_Occ
                            B_Occ -->|Patient Discharged / Switched| B_Clean
                             B_Clean -->|Housekeeping Marks Clean| B_Avail
                                B_Avail -->|Equipment Breakdown| B_Maint
                                     B_Maint -->|Repaired| B_Avail
                                                end

                            subgraph AdmissionWorkflow["Inpatient ADT Operations"]
                        AdmitReq([Admit Patient Request]) --> CheckBed{Is Target Bed Available?}
                            CheckBed -- No --> PickOther["Select Alternative Available Bed"]
    CheckBed -- Yes --> CreateAdmission["Create Admission Record (Status: admitted)\n- Assign Attending Physician\
                            CreateAdmission --> SetBedOcc["Update Bed status to 'occupied'"]

                                 SetBedOcc --> InpatientStay{"Inpatient Stay Actions"}

    InpatientStay -->|Transfer Bed Request| CheckTransferAuth{"Authorized?\n(Attending Doctor / Admin / Receptioni
                           CheckTransferAuth -- Yes --> ValidateNewBed{Target Bed Available?}
    ValidateNewBed -- Yes --> DoTransfer["1. Old Bed -> 'cleaning'\n2. New Bed -> 'occupied'\n3. Update Admission

    InpatientStay -->|Discharge Request| CheckDischargeAuth{"Authorized?\n(Attending Doctor / Admin / Receptionist
    CheckDischargeAuth -- Yes --> CompleteDischarge["1. Set Admission status -> 'discharged'\n2. Record discharge_

    CompleteDischarge --> GenInpatientInvoice["Generate Inpatient Invoice:\n- Daily Room Rate x Days\n- Attending
              GenInpatientInvoice --> NotifyPatientDischarge["Send Discharge Notification to Patient"]
                                                end
```

## 5. Emergency Ambulance Booking & Live GPS Tracking Flow

Patients or staff can locate the nearest ambulance on a live interactive map, dispatch emergency transport, and track the vehicle in real-time with simulated GPS waypoints.

```
flowchart TD
StartAmbulance([User Opens /ambulance]) --> GetLocation["Detect User Device GPS\n(or Default to Apex Horizon Coord
GetLocation --> FetchFleet["Query Fleet Ambulances & Filter by Distance / Radius"]
FetchFleet --> DisplayMap["Render Interactive Map with Leaflet / OpenStreetMap\n- Available, Dispatched, In-Transi

DisplayMap --> ClickBook["Submit Booking (/ambulance/book)\n- Input Contact Phone, Address, Destination Dept\n- Au

ClickBook --> DispatchEngine{"Ambulance Selection"}
DispatchEngine -->|Specific Ambulance Chosen| VerifyAvailable{Is Vehicle Available?}
DispatchEngine -->|No Preference| FindNearest["Run Haversine Distance Calculation\nFind Nearest Available Ambulanc

VerifyAvailable -- No --> FindNearest
FindNearest --> AnyAvailable{Any Ambulance Available?}
AnyAvailable -- No --> Alert911["Display Alert: All Units Busy.\nAdvise calling emergency 911."]

AnyAvailable -- Yes --> CreateBooking["Create AmbulanceBooking Record:\n- Status: 'assigned'\n- Link Patient & Ass
CreateBooking --> UpdateVehicle["Update Ambulance status -> 'dispatched'\nLog First GPS Coordinate in ambulance_lo
UpdateVehicle --> RedirectLiveTrack["Redirect to Live Tracking Screen (/ambulance/track/{id})"]

subgraph LiveTracking["Real-Time Tracking & Simulation Loop"]
RedirectLiveTrack --> LoadTrackUI["Render Zomato-Style Live Tracking UI\n- Live Vehicle Marker, Patient Marker

LoadTrackUI --> PollPosition["Client Polling / SSE: /ambulance/api/track/{id}"]
PollPosition --> MovementSim["Driver/Admin GPS Simulation Step:\n/ambulance/api/simulate-step/{id}"]

MovementSim --> CalcStep["Interpolate Waypoint towards Pickup / Hospital\nUpdate current_latitude & current_lo
CalcStep --> CheckArrive{Arrived at Destination?}

CheckArrive -- No --> PollPosition
CheckArrive -- Yes --> CompleteTrip["Set Booking status -> 'completed'\nSet Ambulance status -> 'available'\nC
end
```

## 6. Pharmacy Inventory & Medication Dispensing Flow

Doctors issue digital prescriptions that flow directly into the pharmacy queue. Pharmacists dispense medication, automatically decrementing physical inventory and alerting low-stock levels.

```
flowchart TD
    DocIssues([Doctor Issues Prescription]) --> ValidateStock{"Check Medicine Stock in DB"}
ValidateStock --> SaveRx["Save Prescription (Status: 'active')\nCreate PrescriptionItems with dosage, frequency, d
    SaveRx --> RxQueue["Prescription Enters Pharmacy Dispense Queue (/pharmacy/dispense-queue)"]

    RxQueue --> PharmacistInspects["Pharmacist Reviews Prescription (/prescriptions/{id})"]
    PharmacistInspects --> ClickDispense{"Click 'Dispense Medication'"}

    ClickDispense --> CheckAuth{"User is Pharmacist / Staff?"}
    CheckAuth -- No --> RejectAction["403 Unauthorized Action"]
CheckAuth -- Yes --> DeductInventory["Loop through PrescriptionItems:\nUpdate Medicine stock_quantity = stock_quar

    DeductInventory --> CheckReorder{"stock_quantity <= reorder_level?"}
  CheckReorder -- Yes --> TriggerLowStock["Flag Medicine: 'Low Stock Alert'\nDisplay in Superadmin Dashboard"]
    CheckReorder -- No --> MarkDispensed["Update Prescription Status -> 'dispensed'"]
    TriggerLowStock --> MarkDispensed

    MarkDispensed --> LogAudit["Create DISPENSE Audit Log"]
  LogAudit --> PrintLabel["Print Patient Medication Bag / Instruction Sheet"]
```

## 7. Diagnostic Laboratory & Pathology Workflow

Diagnostic test orders progress from initial doctor request to laboratory specimen collection, testing, result recording, and instant reporting.

```
stateDiagram-v2
    [*] --> Requested: Doctor Orders Diagnostic Test (/lab/order)
Requested --> InProgress: Lab Technician Receives Request (/lab/requests)

    state InProgress {
  [*] --> SampleCollection: Collect Specimen (Blood/Urine/Biopsy/Imaging)
    SampleCollection --> Processing: Run Pathology / Radiologic Analysis
    Processing --> Entry: Technician Inputs Findings & Result Summary
    }

InProgress --> Completed: Submit Final Results (/lab/requests/{id}/result)
    InProgress --> Cancelled: Test Cancelled / Specimen Unusable

    Completed --> ReportReady: Generate Standardized Lab Report (#REP-XXXX)
ReportReady --> NotifyDoctorPatient: Send Instant System Notification to Doctor & Patient
    ReportReady --> [*]
```

## 8. Billing, Contactless QR Payments & Cashier Verification Flow

Supports both traditional front-desk cashier payments and modern mobile camera contactless QR scan payments with two-step receptionist verification.

```
sequenceDiagram
    autonumber
    actor Patient as Patient
    participant QR as Mobile QR Scanner
    participant Sys as Hospital Server
    actor Reception as Receptionist / Cashier
    participant Audit as Audit Trail

    alt Scenario A: In-Person Cashier Counter Payment
    Patient->>Reception: Visits Cashier Counter with Invoice #INV-XXXX
    Reception->>Sys: Record Offline Payment (/billing/{id}/pay)
    Sys->>Sys: Create Payment (status: 'completed', method: Cash/Card)
    Sys->>Sys: Update Invoice (status: 'paid' or 'partial')
    Sys->>Patient: Send Payment Received Notification & Print Receipt
    else Scenario B: Contactless Mobile QR Scan Payment
    Patient->>QR: Scan Invoice QR Code with Phone Camera
    QR->>Sys: Hit /billing/{id}/qr-scan
    Sys->>Sys: Create Payment (status: 'pending', method: 'upi', ref: 'QR-SCAN-XXXX')
    Sys->>Sys: Set Invoice status -> 'checking'
    Sys->>Reception: Dispatch Real-Time Notification: "Payment Verification Needed"

    Note over Reception,Sys: Verification by Receptionist / Admin
    Reception->>Sys: Review Payment (/billing/{id})

    alt Approved
    Reception->>Sys: POST /billing/{id}/verify (action: 'approve')
    Sys->>Sys: Set Payment status -> 'completed'
    Sys->>Sys: Recalculate Totals & Update Invoice -> 'paid'
    Sys->>Audit: Log Payment Approval
    Sys->>Patient: Notify: "Your QR payment has been confirmed!"
    else Declined
    Reception->>Sys: POST /billing/{id}/verify (action: 'decline', reason: '...')
    Sys->>Sys: Set Payment status -> 'declined'
    Sys->>Sys: Revert Invoice status -> 'unpaid'
    Sys->>Patient: Notify: "Payment declined. Reason: [Reason]"
    end
    end
```

## 9. Driver Duty & Ambulance Maintenance Complaint Flow

Drivers monitor their dispatched vehicles and file maintenance or mechanical complaints that transition through maintenance lifecycle states.

```
flowchart TD
    DriverLogin([Driver Logs In]) --> DriverDash["Driver Dashboard (/dashboard)\n- Shows Assigned Ambulance\n- Duty St

    DriverDash --> EncounterIssue["Vehicle Breakdown / Maintenance Needed\n(e.g., Siren Failure, Brake Pad Wear, Engir
    EncounterIssue --> FileComplaint["Submit Maintenance Complaint (/ambulance-complaints)\n- Vehicle ID, Priority (lo

    FileComplaint --> CompSubmitted["Complaint Status: 'submitted'"]
    CompSubmitted --> AdminReview{"Fleet Admin / Superadmin Review"}

    AdminReview --> SendMaintenance["Send Vehicle to Workshop\n- Update Complaint status -> 'in_maintenance'\n- Update

    SendMaintenance --> RepairsDone["Mechanic Completes Servicing"]
    RepairsDone --> ResolveComplaint["Admin Marks 'resolved' with resolution notes\n- Update Complaint status -> 'reso

    ResolveComplaint --> CloseLoop["Vehicle Ready for Dispatches & Driver Notified"]
```

## 10. End-to-End Cross-Module Entity Relationship Flow

How core database entities interact throughout a patient's lifecycle in the hospital ecosystem:

```
erDiagram
    USERS ||--o{ ROLES : "assigned via role_user"
    USERS ||--o| DOCTORS : "profile"
    USERS ||--o| PATIENTS : "profile"
    USERS ||--o| STAFF : "profile"
    USERS ||--o| AMBULANCE_DRIVERS : "profile"

    PATIENTS ||--o{ APPOINTMENTS : "books"
    DOCTORS ||--o{ APPOINTMENTS : "attends"
    DEPARTMENTS ||--o{ APPOINTMENTS : "hosts"

    APPOINTMENTS ||--o| MEDICAL_RECORDS : "generates"
    PATIENTS ||--o{ MEDICAL_RECORDS : "history"
    DOCTORS ||--o{ MEDICAL_RECORDS : "author"

    MEDICAL_RECORDS ||--o{ PRESCRIPTIONS : "contains"
    PRESCRIPTIONS ||--|{ PRESCRIPTION_ITEMS : "details"
    MEDICINES ||--o{ PRESCRIPTION_ITEMS : "dispensed"

    MEDICAL_RECORDS ||--o{ LAB_REPORTS : "triggers"
    LAB_TESTS ||--o{ LAB_REPORTS : "specifies"

    PATIENTS ||--o{ ADMISSIONS : "admitted as inpatient"
    BEDS ||--o{ ADMISSIONS : "assigned to"
    ROOMS ||--|{ BEDS : "contains"
    DEPARTMENTS ||--|{ ROOMS : "belongs to"

    PATIENTS ||--o{ AMBULANCE_BOOKINGS : "requests emergency transport"
    AMBULANCES ||--o{ AMBULANCE_BOOKINGS : "dispatched for"
    AMBULANCE_DRIVERS ||--o{ AMBULANCE_BOOKINGS : "operates"
    AMBULANCE_BOOKINGS ||--o{ AMBULANCE_LOCATION_LOGS : "GPS breadcrumbs"
    AMBULANCES ||--o{ AMBULANCE_COMPLAINTS : "maintenance logs"

    APPOINTMENTS ||--o| INVOICES : "billed via"
    ADMISSIONS ||--o| INVOICES : "billed via"
    INVOICES ||--|{ INVOICE_ITEMS : "itemized lines"
    INVOICES ||--o{ PAYMENTS : "paid via offline or QR"
    USERS ||--o{ NOTIFICATIONS : "receives"
    USERS ||--o{ AUDIT_LOGS : "audited by"
```

## 🚀 Summary of Key Technical Highlights

- **Framework**: Laravel 11 with Eloquent ORM & Blade Templates.
- **Dynamic Dashboards**: Centralized routing in `DashboardController` directing each user to customized interfaces.
- **Audit Logging**: Comprehensive activity tracking via `AuditService::log()` across every clinical, financial, and operational event.
- **Real-Time Simulation**: API-driven vehicle movement simulation with Haversine distance computations and breadcrumb logging.
- **Contactless QR Verification**: Asynchronous two-step payment workflow bridging patient mobile scanning with front-desk cashier approvals.



Syntax error in text

mermaid version 11.16.1